T

TPOS

THE OPERATING SYSTEM FOR TRAILER DEALERS

Functional Specification Proposal

Companion ② of three. For each requirement in the **Product Specification**, this document echoes the requirement and proposes a concrete technical solution — architecture, data, API, frontend, real-time, scaling, security, integrations and accounting export — with the reasoning and the roads not taken.

Document v1.0 · **Non-normative** — the Specification states the requirements; everything here is the proposed answer, and the build team owns the final call · Companions: ① Product Specification · ③ Bid & Delivery Pack · Confidential draft

Contents

1. How to Read This Document
2. Proposed Architecture at a Glance
3. Data & Persistence — Tenant Isolation
4. API Design
5. Real-Time Updates
6. Frontend & UX
7. Performance & Scaling
8. Security Mechanisms
9. Integrations
10. Accounting Export
11. Architecture Decisions & Roads Not Taken
12. Requirement → Proposal Traceability

1 · How to Read This Document

The **Product Specification** is deliberately technology-neutral: it states *what* must be true. This Proposal answers each of those requirements with a *proposed* technology and design, plus the reasoning. The pattern throughout:

> **Requirements echoed.** Each section opens with a box like this one, quoting the Specification sections it answers — so every proposal is traceable to a requirement, and §12 closes the loop with a full traceability table.

- **Advisory, not mandated.** A rebuild team may substitute equivalents (the Specification is the contract; this document is the recommended route). Decisions that were genuinely contested — including three we reversed when the evidence said so — are recorded honestly in §11.
- **Machine-readable companions:** `schema.sql` (the reference PostgreSQL DDL, kept reconciled with the Specification's data-model requirements) and `openapi.yaml` (the API contract, 66 paths). Both live beside this document.
- **Numbers are order-of-magnitude.** Benchmark figures cited here come from practitioner and vendor sources (§11.2) — use them for intuition and *benchmark your own schema and queries* before treating any as a constant.

2 · Proposed Architecture at a Glance

> **Requirements echoed.** Spec §2 (three deployment modes from one product), §12 (isolation, density, portability, availability).

Layer	Proposal	Why (short)
Database	**PostgreSQL** (hosted) · **SQLite** per self-host tenant behind one data-access layer	Intensely relational domain; transactions + joins; RLS, jsonb, partitioning
Tenant isolation	**Shared schema + Row-Level Security**, `tenant_id` on every table	Database-enforced; survives app bugs; ~1–5% when indexed (§3, §7)
Topology	**Modular monolith** — one deployable	Self-host wants one artifact; no network hops on DB-bound paths
App framework	**Django/DRF** (pragmatic lead) or **NestJS + Next.js** (JS-first)	Batteries-included CRUD, migrations, RBAC, admin
Connections	**PgBouncer** transaction pooling, ~3–5 backend conns/core	The tenant-density lever (§7.3)

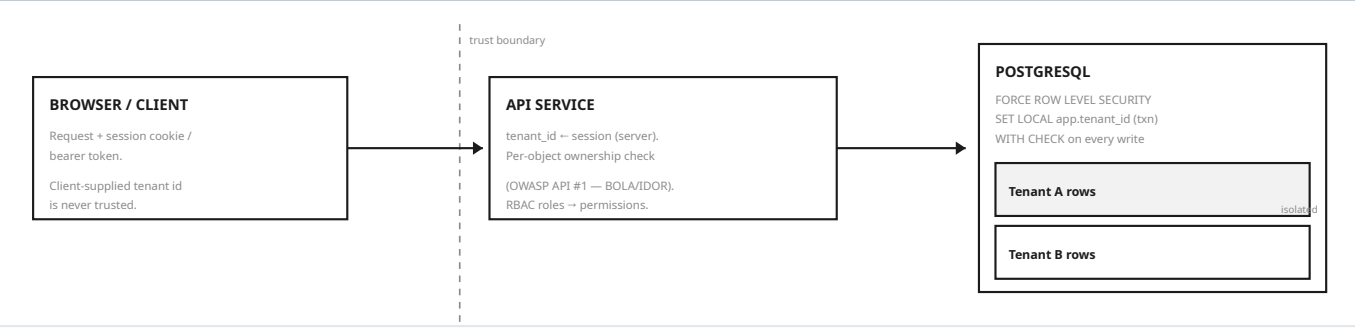
Reads	**Materialized read models** (CQRS-lite) + replicas + cache	Dashboards become O(1) indexed reads (§7.4)
Real-time	DB change feed → pub/sub → **server-push (SSE/WebSocket)**	No polling; tenant-scoped topics (§5)
Frontend	**JS-first** (Next.js / htmx); SSR + edge-cached storefront; WASM modules for hot logic	Component ecosystem + SEO; §6 has the full reasoning incl. the Rust/WASM alternative
Media	**S3-compatible object storage** (MinIO self-host)	Photos/docs out of the DB
Scaling path	Single primary + replicas → HASH(`tenant_id`) partitions → defer sharding	Earn each step (§7.7)

Deployment modes: **hosted** runs the monolith multi-tenant behind the pooler with the operator control plane enabled; **self-host** ships the same artifact configured single-tenant (SQLite or local Postgres), control plane disabled, license-key validated. No tenant feature may assume operator-only services (Spec §12 portability).

3 · Data & Persistence — Tenant Isolation

> **Requirements echoed.** Spec §12 (strict per-tenant isolation at the data layer, provable by an automated negative test), Appendix A (logical data dictionary), Appendix B.5–B.6 (isolation outcomes; audit log, consent records, residency, retention), Appendix C (import idempotency), Appendix D (five date clocks, cost components).

The reference DDL is `schema.sql` — PostgreSQL 15+, generated from Appendix A and kept reconciled with the later appendices. The isolation mechanism, in depth:



Defense in depth for the #1 risk — cross-tenant data disclosure. Tenant context is derived server-side, enforced at the API (object-ownership check) and again in the database (row-level security). Neither layer trusts a client-supplied tenant id.

- **Row-Level Security on every tenant table**, `ENABLE` + `FORCE` (even the table owner is policed), with a `tenant_isolation` policy: `USING / WITH CHECK (tenant_id = current_setting('app.tenant_id', true)::uuid)` — reads filter and writes are constrained to the acting tenant.
- **`SET LOCAL app.tenant_id` per transaction** — transaction-scoped, so it is safe under PgBouncer transaction pooling (a session-level `SET` silently leaks across pooled clients; §7.3).
- The app connects as a **non-owner, non-superuser, non-`BYPASSRLS`** role; migration lint asserts every tenant table has RLS enabled *and* a policy.
- **CI negative test** (the Spec's provability requirement): seed tenants A and B; assert A cannot read/update/delete B's rows and cannot insert rows as B — at the SQL layer *and* through the API; any leak fails the build.
- **Spec-driven schema features:** `external_ref` + per-tenant unique partial indexes on units/customers/leads/deals (import idempotency); append-only `activities` (audit) and `consent_records` (e-sign evidence) with `UPDATE/DELETE` revoked; customer address incl. `state` (per-state breach fan-out); `retention_policies` (legal floors vs the 2-year disposal default); the five unit date clocks + `recon_cost` / `transport_cost`; deal gross-input columns; `export_batches` / `export_batch_items` (accounting idempotency, §10).
- Self-host parity: same DDL; the single-tenant install keeps `tenant_id` columns (schema parity for upgrades) and may skip the policy block.

4 · API Design

> **Requirements echoed.** Spec §6 (role workspaces act on shared records), §8 (cross-cutting: search, notifications, tasks, audit), §10 (role-based authorization on every action), Appendix C (bulk import, idempotent on a stable external reference), Appendix F (on-demand accounting export). The machine-readable contract is `openapi.yaml` (66 paths, OpenAPI 3.1, validated) — the listing below is the human-readable map.

REST / JSON. **Namespaces:** `/api/v1/*` authenticated back-office (tenant from session/subdomain, every query tenant-scoped) · `/api/v1/portal/*` customer portal · `/api/public/*` storefront (no auth, tenant by host) · `/api/platform/*` operator control plane (super-admin, cross-tenant) · `/api/webhooks/*` inbound integrations. RBAC per endpoint (the §10 matrix); state-machine changes use explicit `…/transition` actions (validated + audited), not arbitrary PATCH; lists accept `?q=&status=&sort=&page=&limit=`; money/transition writes accept an `Idempotency-Key` header.

Auth & session

```
POST  /api/v1/auth/login            GET  /api/v1/auth/me
POST  /api/v1/auth/logout           POST /api/v1/auth/sso/callback
POST  /api/v1/auth/password/reset
```

Inventory & pricing

```
GET   /api/v1/units                 list/filter (type,status,location,aged)
POST  /api/v1/units                 intake / acquire
GET   /api/v1/units/:id             full record ("all lenses")   PATCH /api/v1/units/:id
POST  /api/v1/units/:id/transition  incoming→recon→available→pending→sold (guarded)
POST  /api/v1/units/:id/photos      GET /api/v1/units/:id/timeline    GET /api/v1/units/aging
PATCH /api/v1/units/:id/price       set list/floor (logged; may require approval)
GET   /api/v1/pricing/repricer      GET /api/v1/pricing/comps?unit=:id
GET   /api/v1/pricing/approvals     POST /api/v1/pricing/approvals/:id/decide
```

Leads, customers, deals, quotes

```
GET/POST /api/v1/leads              GET/PATCH /api/v1/leads/:id
POST  /api/v1/leads/:id/assign      POST /api/v1/leads/:id/convert  →deal
GET   /api/v1/sources               GET/POST /api/v1/campaigns
GET/POST /api/v1/customers          GET/PATCH /api/v1/customers/:id
GET/POST /api/v1/deals              GET/PATCH /api/v1/deals/:id
POST  /api/v1/deals/:id/stage       advance pipeline
POST  /api/v1/deals/:id/discount    POST /api/v1/deals/:id/approve  (manager RBAC)
GET/POST /api/v1/quotes             POST /api/v1/quotes/:id/send | /accept
GET   /api/v1/forecast              GET /api/v1/team/leaderboard
```

Service · accounting · titles · delivery

```
GET/POST /api/v1/workorders         POST /api/v1/workorders/:id/status
PATCH /api/v1/workorders/:id/tasks/:t   POST …/hours  …/parts  …/photos
GET/POST /api/v1/invoices           POST /api/v1/invoices/:id/payments (idempotent)
GET/POST /api/v1/bills              GET /api/v1/accounting/pnl?period=   POST /accounting/export
GET/PATCH /api/v1/titles/:unitId    GET/POST /api/v1/documents   POST /documents/:id/sign
GET/POST /api/v1/deliveries         POST /api/v1/deliveries/:id/status   POST /deliveries/quote
GET/POST /api/v1/appointments
```

Cross-cutting & tenant admin

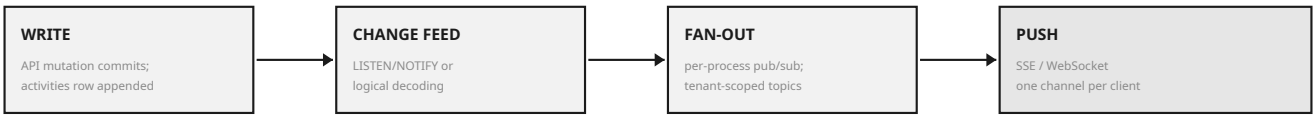
```
GET   /api/v1/search?q=             GET /api/v1/notifications  POST …/:id/read
GET/POST /api/v1/tasks              GET /api/v1/activity        GET /api/v1/reports/:kpiSet
GET/POST /api/v1/users              PATCH /api/v1/users/:id (roles, lot scope)
GET/PATCH /api/v1/tenant/settings   branding · lots · config · billing
```

Storefront (public) & customer portal

```
GET   /api/public/inventory         available units (filter/search/sort)
GET   /api/public/inventory/:stock  detail (deep-linkable)
POST  /api/public/inquiry           creates a Lead + notifies   ← implemented in prototype
POST  /api/public/financing/prequalify
GET   /api/v1/portal/me             GET /api/v1/portal/deals
GET   /api/v1/portal/quotes/:id     POST …/accept
POST  /api/v1/portal/financing/apply  GET …/status
GET   /api/v1/portal/delivery/:id   GET /api/v1/portal/documents  POST …/:id/sign
GET/POST /api/v1/portal/saved
```

Platform control plane (hosted) & webhooks

```
GET/POST /api/platform/tenants      PATCH /api/platform/tenants/:id (plan/status)
GET   /api/platform/revenue         GET /api/platform/usage    GET /api/platform/billing
GET   /api/platform/support         GET /api/platform/health   POST /api/platform/license/verify
POST  /api/webhooks/financing       lender decision → Deal/portal
POST  /api/webhooks/payments        settled → reconcile invoice
POST  /api/webhooks/freight         carrier status → delivery
POST  /api/webhooks/syndication     listing-site lead → Lead
POST  /api/webhooks/title           DMV/title callback
```



Same events invalidate the materialized read models (§7.4) and the storefront edge cache (§7.5). Clients re-fetch entities via the normal RLS-scoped API.

> **Requirements echoed.** Spec §12: changes by one user — status, price, a new lead, an assignment, a payment — appear to others viewing the same data within ~1–2 seconds, **without a manual refresh and without perceptible polling delay**.

Server push, not client polling. Committed writes emit change events; a per-process fan-out pushes them to subscribed clients over one long-lived channel per session, topics scoped by tenant. The same events invalidate the materialized read models (§7.4).

- **Channel:** one long-lived connection per signed-in client — **SSE** (`GET /api/v1/events` , `text/event-stream`) as the default for its simplicity/proxy-friendliness; upgradeable to WebSocket if bidirectional needs appear. A `cursor` query param replays missed events after a reconnect.
- **Feed:** committed mutations write the `activities` row (already required for audit) and emit a change event — implementation options: Postgres `LISTEN/NOTIFY` (simple, fine at this scale) or logical-decoding/outbox if delivery guarantees must tighten. Events carry `{topic, entity_type, entity_id, action, at}` — clients re-fetch the entity through the normal (RLS-scoped) API rather than trusting payload data.
- **Scoping:** topics are derived server-side from the session's tenant + role — the channel can never subscribe across tenants (same rule as §3).
- **Density interaction:** thousands of long-lived connections favor an async runtime (§7.6) and are exactly why the API tier holds no per-request DB connection between events (§7.3).
- **UI behavior:** lists/kanbans patch in place (a tile moves, a row updates, a badge increments); the storefront does *not* hold channels — its freshness comes from edge-cache invalidation on status change (§7.5).

6 · Frontend & UX

> **Requirements echoed.** Spec §6 (twelve role workspaces over shared records), §7 (mobile-first, SEO-visible storefront), §12 (responsive phone → tablet → laptop → ultra-wide; WCAG 2.1 AA; field roles usable on a phone; dashboards < 2 s; real-time UI).

6.1 Recommendation — JS-first, with WASM where it earns its keep

- **UI stack: JS/TS. Next.js** (React) as the default — or **Django + htmx** for the back office if the Django profile is chosen in §2. Rationale: the back office is forms-, tables- and CRUD-heavy, which is where the JS ecosystem's mature component libraries (data grids, date pickers, form kits) pay off daily; hiring is easiest; SSR for the storefront is first-class.
- **Storefront: DOM-rendered SSR + edge cache** (§7.5) — the public site must be crawlable and fast on a phone; canvas-rendered approaches are ruled out by SEO/accessibility alone.
- **Rust→WASM as modules, not the whole app:** the derived-field formula engine (Spec Appendix D — margins, aging, commissions) is a natural WASM module: pure, correctness-critical, shared verbatim between server and client so previews always match bookings.
- **Responsive strategy:** one responsive web app (no separate native build in Phase 1 — see the Bid Pack); layouts collapse from the ultra-wide dashboard grid → laptop → tablet two-pane → phone single-pane with bottom navigation; recon & delivery screens are touch-first since they're used in the yard; WCAG 2.1 AA with full keyboard navigation.

6.2 The Rust/WASM alternative — assessed honestly

If the build team is Rust-first by conviction, the only whole-app WASM options that preserve SEO/accessibility are the DOM-rendered frameworks — **Leptos** (web-first: SSR, islands, fine-grained signals, the strongest Rust web story) or **Dioxus** (one codebase across web/desktop/mobile; React-like). Choose Leptos for this product if at all — the storefront is a web/SSR problem. Accept the trade-offs: component ecosystems are young (Thaw UI / headless primitives + Tailwind, no AG-Grid-class data grid), APIs move fast, and the hiring pool is small. Canvas-rendered stacks (egui, Flutter Web) are **not** acceptable for the storefront or the accessible back office.

6.3 Data tables & filtering — server-side by design

No framework choice changes this: with tens of thousands of rows per tenant, **filtering/sorting/pagination belong in the API and SQL**, not the browser. Every list endpoint takes `?q=&status=&sort=&page=&limit=` (already in `openapi.yaml`); the table component stays "dumb" and renders a page; the filter UI (facet chips, date ranges, saved views) is bespoke in any stack. This sidesteps the one place the Rust UI ecosystem is furthest behind (rich client-side data grids) and keeps payloads small on yard Wi-Fi.

7 · Performance & Scaling

> **Requirements echoed.** Spec §12: interactive actions feel instant; dashboards < 2 s; **maximize simultaneous active users per server**; hundreds+ tenants and tens of thousands of units/leads per tenant with no per-tenant degradation; 99.9% availability with a cache-served storefront.

TPOS is **database-bound CRUD**: requests spend their time waiting on Postgres, not on application code. The performance and tenant-density wins therefore live in the **data layer and the connection pool**, not in an exotic runtime or premature sharding. This appendix is one optimization profile; apply it only to the degree the product needs.

Define the dimension first. "High performance" is not one thing, and the lever order changes with it. Decide which of these dominates before optimizing — over-investing in the wrong one is wasted complexity.

If the goal is…	Dominant levers, in order
p99 read latency (dashboards feel instant)	Materialized read models → cache hot reads → `tenant_id`-leading indexes → read replicas
Write / transaction throughput	Index discipline → prepared statements & batching → hash-partition hot tables → pool sizing
Tenants per server (density)	PgBouncer transaction pooling → edge-offload the storefront → shared-schema low footprint → async runtime

7.1 Indexing & query design — the foundation (and what makes RLS cheap)

- Every hot index **leads with `tenant_id`**, then the query's predicates: `(tenant_id,status)`, `(tenant_id,stock_no)`, `(tenant_id,due_at)`, `(tenant_id,stage)`, etc.
- **Keep Row-Level Security.** With the right composite index the planner pushes the policy predicate into the index scan, so RLS is cheap (reported order-of-magnitude: ~0.3 ms policy evaluation at 50 M rows / 10 K tenants; ~1–5% vs. app-level filtering). **The 100× penalty comes from a missing `tenant_id`-leading index, not from RLS itself.** Database-enforced isolation is worth the few percent — one cross-tenant leak is existential, and it protects you even when application code has a bug.
- Eliminate N+1s; use prepared statements; keep working sets index-resident.

7.2 Partitioning — secondary, and use HASH not list

- Partitioning is a lever only for the largest tables (e.g. `activities`, `units` at very high volume); the composite index above does most of the work first.
- With thousands of tenants, partition by **`HASH(tenant_id)` into a fixed number of partitions** (e.g. 16–64) for cache locality and scan pruning. **Do NOT list-partition one-per-tenant** — thousands of partitions blow up planner time and DDL. Time-range sub-partitioning suits the audit log.

7.3 Connection pooling — the real density lever

- Postgres is process-per-connection (~5 MB + context-switch per connection), so density comes from **PgBouncer in transaction-pooling mode** multiplexing many clients onto a small pool sized to roughly **3–5 backend connections per CPU core**.
- **Gotcha that interacts with RLS:** under transaction pooling a session-level `SET app.tenant_id` does not persist across statements. Set the tenant **per-transaction** with `SET LOCAL app.tenant_id = …` (or `set_config('app.tenant_id', …, true)`) so RLS still applies. Also avoid session-scoped prepared-statement / session-feature assumptions in transaction mode.

7.4 Read path — precompute the dashboards (CQRS-lite)

- The KPI dashboards (§9) aggregate across many rows. Precompute them into **materialized views / rollup tables** refreshed incrementally by the background worker, so a dashboard load is an O(1) indexed read, not a per-request fan-out join. This is the single biggest internal-latency win.
- Cache hot reads (Redis / Dragonfly or in-process) with event-driven invalidation on writes; route all dashboard/report traffic to **read replicas**.

7.5 Storefront at the edge

The public storefront's "available inventory" is the highest-volume, most cacheable traffic. Serve it from a **CDN / ISR edge**, invalidated when a unit's status changes. This removes the largest read load from the origin entirely and is often the cheapest way to raise effective tenants-per-server.

7.6 Runtime — a minor lever for DB-bound work

Framework benchmarks show per-request gaps **collapse once Postgres is in the loop** (compiled runtimes dominate only the compute-bound plaintext/JSON rounds). For DB-bound CRUD you will not get throughput from switching language — choose the runtime for its **concurrency model and your team**, not per-query speed. Where it touches density: an async / light-concurrency runtime (**Go, .NET, Node, or Elixir** for connection-heavy/real-time) holds more in-flight requests per box than a process-per-worker Python/Ruby setup. **Rust is not required** here and its productivity cost rarely pays back on this workload.

7.7 Scaling ladder — earn each step

1. Vertical scaling + PgBouncer (handles a large share of workloads on its own).
2. Add read replicas for the read/report load.
3. Hash-partition the few largest tables.
4. **Only** once a single primary is genuinely exhausted (commonly hundreds of GB+), shard with **Citus** co-located by `tenant_id`. Keep the schema partition-ready (`tenant_id` on every table) so this migration is cheap — but **defer it**; vanilla Postgres + replicas goes much further than teams expect.

> **Caveat — treat the numbers as order-of-magnitude.** The RLS timing figures come from a single practitioner benchmark and the single-node size thresholds are partly vendor framing. Use them for intuition; **benchmark on your own schema, queries and data** before treating any figure as a constant. Sources:

practitioner RLS-at-scale benchmarks, PgBouncer pooling guides (connection-cost & per-core sizing), single-node-Postgres scaling write-ups, and the TechEmpower framework benchmarks (database-query rounds).

8 · Security Mechanisms

> **Requirements echoed.** Spec Appendix B.5 states the security *outcomes* (isolation, hardened sessions, strong password storage, MFA, throttling, token hygiene, CSRF defense, secrets, encryption, immutable audit, rate limits, validated uploads, SSRF restraint). These are the proposed *mechanisms*.

Identity & sessions

- Passwords: **Argon2id** (m = 19 MiB, t = 2, p = 1); bcrypt ≥ 10 accepted for legacy migration only.
- Cookies: `HttpOnly` + `Secure` + `SameSite`, `__Host-` prefix; session id ≥ 64-bit entropy; rotate on login; idle + absolute timeouts; server-side logout.
- **MFA** (TOTP/WebAuthn) — required for admin, titles and payment-touching roles.
- Login throttling + lockout; reset tokens single-use, time-limited, no account enumeration.
- Portal/API tokens: short-lived, role-scoped, signature/issuer/audience/expiry all validated.
- CSRF: synchronizer token (or HMAC double-submit) on cookie-auth routes; custom-header requirement on the bearer API.

Platform & data

- Isolation mechanism per §3 (RLS `FORCE`, `SET LOCAL`, non-owner role, CI negative test).
- Authorization: **RBAC** (role → permission sets, the Spec §10 matrix) layered over **ABAC** (tenant id + field sensitivity) with field-level masking of PII / title / payment data.
- Secrets in a managed KMS/secret store; never in VCS; DB + token-signing keys rotated.
- TLS in transit; encrypted volumes at rest; column-level encryption/tokenization for payment tokens & sensitive PII (only token + last4 + brand are ever stored — §9).
- Audit: append-only `activities` (UPDATE/DELETE revoked) with actor, action, entity, timestamp.
- Rate limits per IP / token / tenant on auth, API, upload. Server-side validation + parameterized queries everywhere.
- Uploads: magic-byte content verification (never trust Content-Type), randomized names, object storage off-webroot, malware scan, image re-encode.
- SSRF: server-side fetches allow-listed; RFC1918 / link-local / metadata addresses blocked; resolved IP validated.

The compliance regime these mechanisms serve (GLBA Safeguards Rule, breach reporting, ESIGN, retention) is specified — with citations — in Spec Appendix B; the annual pen test + vulnerability-assessment cadence it requires is carried as a delivery line in the Bid & Delivery Pack.

9 · Integrations

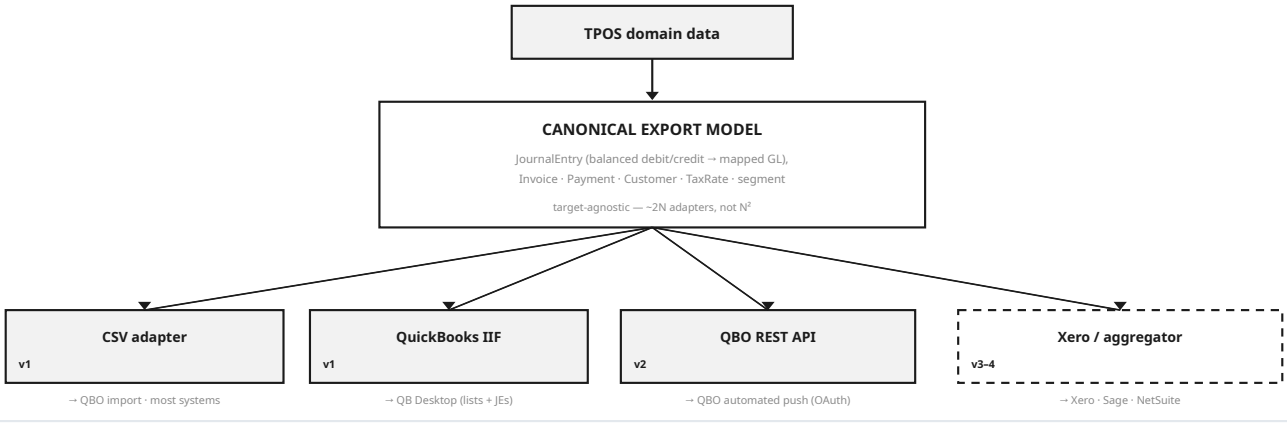
> **Requirements echoed.** Spec §11 names the integration capabilities (financing, listing syndication, title/e-sign, payments, freight, accounting, email/SMS, platform billing); Appendix B constrains how payment data may be handled; Appendix E defines the notification catalog.

Capability	Proposed approach	Key constraint honored
Payments (card + ACH)	**Stripe Connect, direct charges** — each dealer is its own merchant of record; processor-hosted fields/iframe; store token + last4 + brand only	Card data never touches TPOS servers → every dealer stays at **PCI SAQ A**; NACHA: no plaintext bank numbers, WEB-debit account validation, authorizations retained ≥ 2 yrs
E-signature	Hosted e-sign provider (Dropbox Sign / DocuSign class) wrapped by our consent flow	ESIGN consent gate + tamper-evident `consent_records` (hash, signer, timestamp); **titles excluded** — per-DMV paper/ELT
Email / SMS	Transactional providers (Postmark/SendGrid class; Twilio for SMS) behind one notification service driven by the Appendix E catalog	Transactional vs marketing split; opt-out tracking; per-tenant from-domain
Listing syndication	Adapter per channel (TruckPaper, Facebook, …): push *available* units, pull leads back via `/api/webhooks/syndication`	Storefront/status rules in Spec §7 — units appear/disappear with status
Financing / credit pull	Stub-first: manual financing status at MVP; vendor rails (soft-pull, lender submission) added per the Bid Pack phases	FCRA/GLBA scope only enters when the vendor is wired (Spec App. B)
Freight	Manual rate entry at MVP; carrier-API adapter later	—
Platform billing	Subscription provider (Stripe Billing class) driving the operator control plane; usage metering from `usage_metrics`	Hosted mode only (Spec §2)

Vendor choices, build-vs-stub per phase, account ownership and review lead-times are a bid concern — the full register lives in the **Bid & Delivery Pack**.

10 · Accounting Export

> **Requirements echoed.** Spec Appendix F: manual on-demand export of invoices/payments/journal entries; QuickBooks first (Online *and* Desktop); extensible to other systems without re-engineering; one-time chart-of-accounts mapping; no duplicate postings on re-export; baseline must not depend on third-party access or an external approval gate.



Target-agnostic **canonical export model** with per-target **adapters** — the no-rewrite seam. Targets become pluggable serializers (the same architecture a unified-API vendor hosts internally), turning N^2 point-to-point translators into ~2N. File adapters ship first and are *permanent* infrastructure, since QuickBooks Desktop has no clean REST API.

- **Canonical model:** a thin, export-only representation — balanced journal entries (debit/credit lines → mapped GL accounts) plus customer / invoice / payment / tax-rate and a generic "segment" dimension (maps to QBO classes or Xero tracking categories later).
- **v1 — file adapters (ships first):** manual, on-demand **summary journal-entry** export as **CSV** (QuickBooks Online) and **IIF** (Desktop). Needs no OAuth and no Intuit production review; sidesteps QBO's invoice-CSV sales-tax block and 100-row cap. Trade-off accepted: GL-accurate books, but no invoice-level AR aging inside QuickBooks until the API phase.
- **GL mapping wizard (one-time, per dealer):** map system buckets — trailer revenue, COGS, AR, sales-tax payable, fees, customer deposits — to the dealer's own accounts (`GET/PUT /api/v1/accounting/mapping`). File targets need exact account-name matches; the API target resolves and stores QuickBooks Account Ids.
- **Idempotency:** every run creates an `export_batch` ; exported records are marked via `export_batch_items` so a re-run never re-emits them. On the API path, the invoice number becomes QuickBooks `DocNumber` + query-before-insert (QuickBooks has no native idempotency key).
- **Phases 2–4** (triggered by demand, sized in the Bid Pack): QBO REST-API push (OAuth2 + Intuit review) → Web Connector/qbXML for Desktop detail → additional systems via new adapters, or a unified-API aggregator once targets exceed ~2.

11 · Architecture Decisions & Roads Not Taken

Every significant fork weighed for the rebuild — the options considered, what was chosen, what was rejected and why. Decisions are advisory; the build team owns the final call. Several were **revised mid-review as evidence came in** — those are flagged (REVISED) and recorded honestly rather than quietly replaced. Bracketed numbers reference the sources in §11.2.

#	Decision	Options considered	Chosen	Roads not taken — reasoning
1	Prototype persistence	Flat JSON file · real database	JSON file + in-browser mock	The prototype is a UI/IA demo; only the storefront lead endpoint is live. A real DB was out of scope for a front-end mock.
2	Production database	PostgreSQL · document/NoSQL · MySQL · SQLite-only	**PostgreSQL** (hosted) + SQLite per self-host tenant	NoSQL rejected — the domain is intensely relational (one-record-many-lenses = foreign keys), needs transactions (booking a sale) and joins. MySQL viable; Postgres preferred for RLS, jsonb, partitioning, extensions. SQLite-only rejected for hosted (concurrency) but ideal for a single self-host tenant. [1]
3	Multi-tenant isolation	Shared schema + RLS · schema-per-tenant · database-per-tenant · app-level filtering only	**Shared schema + RLS**, `tenant_id` on every table	DB-per-tenant rejected — poor density, heavy ops. Schema-per-tenant rejected — migration/ops complexity at scale. App-filtering-only rejected — one forgotten `WHERE` leaks data; isolation belongs in the database. RLS gives DB-enforced safety at ~1–5% when indexed. [1][2][3][4]
4	Service topology	Modular monolith · microservices	**Modular monolith** (one deployable)	Microservices rejected — the self-host requirement wants one artifact, the team is small, and network hops hurt DB-bound latency. Extract a service later only if a piece proves it needs it.
5	App framework (pragmatic profile)	Django/DRF · Rails · NestJS+Next.js · Go · .NET · Elixir	**Django/DRF** (lead) or **NestJS+Next.js** (JS-first)	Go/.NET/Rust rejected as the default — more boilerplate for CRUD/admin and no free admin plane. Chosen for batteries-included CRUD, migrations, RBAC, Postgres-first tooling, and clean self-host packaging.
6	Media storage	S3-compatible object storage (MinIO self-host) · blobs in Postgres	**S3-compatible** object storage	Blobs-in-DB rejected — bloats the database and slows backups/replication; photos & signed documents belong in object storage.
7	Storefront rendering	Edge/CDN (ISR) · in-app SSR · SPA	**SSR + edge-cache** available inventory	Pure SPA rejected for the public site — SEO & first-paint matter. Internal

				dashboards stay SPA/API (or SSR-with-enhancement).
8	Runtime under *performance-primary* **REVISED**	Rust · Go · .NET · Elixir · keep pragmatic	Async/light-concurrency (**Go/.NET/Elixir**); **Rust not required**	Initially proposed Rust for raw throughput. Evidence: framework gaps **collapse once Postgres is in the loop**; Rust leads only compute-bound rounds. For DB-bound CRUD the runtime is a minor lever — choose for concurrency model + team, not per-query speed. [9]
9	RLS under *performance-primary* **REVISED**	Keep RLS (indexed) · move isolation to app/partitioning for speed	**Keep RLS** + `tenant_id`-leading composite indexes	Initially suggested moving isolation out of RLS for speed. Evidence: RLS ~0.3 ms at 50 M rows / 10 K tenants *with the right composite index*; the 100× penalty is the **missing index**, not RLS. Keep it — the safety is worth ~1–5%. [2][3]
10	Horizontal sharding **REVISED**	Citus early (ease later migration) · single-node + replicas, defer	**Single node + replicas; defer Citus**, keep schema shard-ready	Initially put Citus "on the table." Evidence: vanilla Postgres + pooling handles most workloads under ~500 GB; replicas/partitioning extend into the terabytes before sharding is warranted. Defer until one primary is genuinely exhausted. [5][6]
11	Partitioning	HASH(tenant_id) fixed N · list/partition-per-tenant	**HASH(tenant_id)** into 16–64 partitions (large tables only)	Partition-per-tenant rejected — thousands of tenants blow up planner time & DDL. Partitioning is a *secondary* lever after the composite index. [6]
12	Connection management	PgBouncer transaction pooling · session pooling · none	**PgBouncer (transaction mode)** ~3–5/core + `SET LOCAL` tenant	No/session pooling rejected for density — process-per-connection (~5 MB each). Transaction mode requires `SET LOCAL app.tenant_id` per-transaction so the RLS variable survives (session `SET` silently breaks under it). [7][8]
13	Data access (perf paths)	Heavy ORM · thin typed SQL	**Thin typed SQL** (sqlx/sqlc/Dapper/Ecto) on hot paths	Heavy ORM rejected on hot paths — per-query overhead and hidden N+1s. ORM is fine for cold CRUD.
14	Dashboard read path	Per-request aggregation · materialized read models	**Materialized views / rollups** (CQRS-lite) + replicas + cache	Per-request big-join aggregation rejected for dashboards — precompute and serve an O(1) indexed read.

11.1 What we changed our minds about (and why)

Three decisions from the first "performance-first" pass were corrected once benchmark evidence was reviewed — recorded here so the reasoning is auditable, not lost:

- **#8 — Rust is not the lever.** For DB-bound CRUD, runtime choice barely moves throughput; the language win is real only on compute-bound paths. [9]

- **#9 — Don't drop RLS for speed.** The cost is the missing `tenant_id`-leading index, not the policy; keep DB-enforced isolation. [2][3]
- **#10 — Don't shard early.** A single Postgres primary + replicas + partitioning goes much further than expected; defer Citus. [5][6]

11.2 References & evidence quality

Sources consulted, with their nature flagged. **Several are vendor or single-practitioner sources; all numeric figures are order-of-magnitude guidance — benchmark on your own schema, queries and data before treating any as a constant.**

#	Source	Nature	Informs
1	PostgreSQL docs — Row Security Policies	Official documentation	RLS canonical reference (#2,#3)
2	RLS-at-scale benchmark (DEV Community)	Single-practitioner benchmark	~0.3 ms at 50 M/10 K with `tenant_id`-leading index; missing index = 100× (#3,#9)
3	RLS overhead write-up (Wellally)	Practitioner	~1–5% vs app-level filtering; planner push-down (#3,#9)
4	Tenant-isolation rationale (OneUptime); Crunchy Data RLS-for-tenants	Vendor / practitioner	DB-enforced isolation protects against app bugs (#3)
5	Single-node Postgres scaling (Velodb)	Practitioner / vendor	Vertical + pooling under ~500 GB; OpenAI on Postgres; replicas+partitioning to TB (#10)
6	Citus multi-tenant sharding docs & blog	Vendor framing	Shard by `tenant_id`, co-location; "~100 GB" threshold (#10,#11)
7	Connection scaling (PlanetScale); sizing (Akamai)	Vendor	Process-per-connection cost; PgBouncer multiplexing; ~3–5 conns/core (#12)
8	"PgBouncer … fraught with peril" (JP Camara)	Practitioner deep-dive	Transaction-pooling gotchas; `SET LOCAL` / session-feature caveats (#12)
9	TechEmpower Framework Benchmarks	Independent benchmark	DB-query rounds collapse framework gaps; Rust leads only plaintext/JSON (#8)

12 · Requirement → Proposal Traceability

Where each Specification requirement area is answered. (Spec section / appendix → Proposal section.)

Specification requirement	Answered in
§2 Multi-tenancy & deployment modes · §12 isolation, portability	§2, §3
§6 Role workspaces · §8 cross-cutting · §10 permissions	§4 (API), §6 (UI), §8 (RBAC/ABAC)
§7 Storefront (mobile-first, SEO, lead capture)	§6.1, §7.5
§12 Real-time collaboration (no refresh, no polling delay)	§5
§12 Concurrency & density (max users per server) · performance · scalability · availability	§7
§12 Responsive phone→ultra-wide · accessibility WCAG 2.1 AA	§6.1
Appendix A Data dictionary	§3 + `schema.sql`
Appendix B Security & compliance outcomes	§3, §8 (+ pen-test cadence in the Bid Pack)
Appendix C Onboarding & import (idempotent bulk, error reports)	§3 (`external_ref`), §4 (`/bulk` , `/imports`)
Appendix D Derived-field formulas (one engine, identical results)	§3 (date clocks, cost & gross columns), §6.1 (shared WASM formula module)
Appendix E Notification & document catalog	§5 (in-app), §9 (email/SMS service)
Appendix F Accounting-system export	§10 + `schema.sql` (`export_batches`) + `openapi.yaml` (`/accounting/*`)
§13 Acceptance criteria (end-to-end scenarios)	The integration of all of the above; delivery gates in the Bid Pack

Scope cutline, vendor register, assumptions/RACI, estimate and delivery stages: see the **Bid & Delivery Pack** (companion ③).